

```
# Input data for graph inputs.
x = np.array([[1.0, 2.0, 3.0], [4.0, 5.0, 6.0]])
y = np.array([[3.0, 3.0, 3.0], [3.0, 3.0, 3.0]])

# Use the backend and prepare the ONNX model for execution.
tf_rep = prepare(model)

# Run the tensorflow representation with inputs.
output = tf_rep.run([x, y])

# Output
print("Output:", output)
```

Run the above sample code; it produces the following output, which is a result of adding two input tensors.

```
test@test-pc:~$ python3 test.py
Output: Outputs(out=array([[4., 5., 6.],
        [7., 8., 9.]], dtype=float32))
```

In the above example, we built a simple graph by constructing ONNX nodes. This example uses the TensorFlow back-end for execution. ONNX offers a wide range of back-end support to execute the same ONNX model. The following section explains how to use a converter to import a pretrained graph from a different framework and then execute it.

Using a converter

As its name suggests, ONNX is aimed at exchanging AI models across frameworks. ONNX has a project called 'onnxmltools' (<https://github.com/onnx/onnxmltools>) which has converters for various frameworks like CoreML, Scikit Learn, Keras, SparkML, LightBGM, LibSVM and XGBoost. For TensorFlow models, the project is 'tensorflow-onnx' (<https://github.com/onnx/tensorflow-onnx>), which provides the converter utility.

These converters basically enable converting an AI model from one framework to another. Let's look at the example of importing a pretrained TensorFlow model into ONNX, and then infer and save it in the ONNX format.

Download and extract the Mobilenet pretrained model from http://download.tensorflow.org/models/mobilenet_v1_2018_08_02/mobilenet_v1_1.0_224.tgz.

Extracting this will have a TensorFlow protobuf as shown below:

```
test@test-pc:~$ ls mobilenet_v1_1.0_224
Mobilenet_v1_1.0_224.ckpt.data-000000-of-00001
Mobilenet_v1_1.0_224.ckpt.meta
Mobilenet_v1_1.0_224_frozen.pb
Mobilenet_v1_1.0_224.ckpt.index
```

Use the following sample and load the TensorFlow model, convert to an ONNX model and infer using the ONNX Runtime back-end.

```
# Tensorflow imports
import tensorflow as tf

# Numpy
import numpy as np

#Onnx
from onnx import load

# Tensorflow to ONNX converter
import tf2onnx

# OnnxRuntime backend
import onnxruntime.backend as backend

# Import the Tensorflow protobuf into session
tf.reset_default_graph()
with tf.gfile.FastGFile("./mobilenet_v1_1.0_224_frozen.pb",
'rb') as f:
    graph_def = tf.GraphDef()
    graph_def.ParseFromString(f.read())
    graph = tf.import_graph_def(graph_def, name='')

with tf.Session() as sess:
    # Use tf2onnx converter and build onnx_graph
    onnx_graph = tf2onnx.tfonnx.process_tf_graph(sess.
graph, input_names=["input:0"], output_names=["MobilenetV1/
Predictions/Reshape_1:0"])

    # Make ONNX model from graph
    model_proto = onnx_graph.make_model("test")

    # Optionally the ONNX model can be saved as a
serialized protobuf.
    with open("mobilenet_v1.onnx", "wb") as f:
        f.write(model_proto.SerializeToString())

    # A ONNX saved model can be loaded by load utility.
    model_new = load("mobilenet_v1.onnx")

    # Prepare some random test data
    in_shape = (1, 224, 224, 3)
    data = np.random.uniform(size=in_shape).
astype('float32')

    # We can now use any ONNX backend to execute the
graph
```